

Week 2 · Day 13: Load Testing & Full Observability

Locust, Throughput Benchmarking, OpenTelemetry, Loki, Cost Tracking & A/B Testing

Goal: Your system now has full visibility: traces, logs, metrics, and cost. By end of day you will be able to find any performance regression in under five minutes, attribute every dollar spent to a request, and safely A/B test model changes without full traffic exposure.

Engineering principle: Load test before production, not after an incident. Observability is not optional instrumentation added later -- it is a first-class architectural requirement built in from day one. Every signal you don't instrument today becomes a blindspot during your next 3am incident.

PART A — Locust Load Testing

Why Locust for LLM Load Testing?

Locust is a Python-based load testing framework that models users as coroutines, making it ideal for LLM workloads where requests are long-lived (1-30 seconds each). Unlike tools like wrk or k6, Locust lets you write test scenarios in plain Python with full control over request construction, streaming response consumption, and per-request metric recording -- all essential for LLM benchmarking.

LLM-Specific Load Testing Concerns

Concern	Why it matters for LLMs	What to measure
Concurrency ramp	KV cache fills up -- behaviour changes at scale	Throughput ceiling, OOM threshold
Mixed prompt lengths	Prefill cost is proportional to prompt length	TTFT distribution by prompt length
Streaming vs non-stream	Streaming hides latency but still consumes GPU	TTFT separately from E2E latency
Sustained load (soak)	Memory leaks, block fragmentation accumulate	Latency drift over 30+ minutes
Spike test	Continuous batching must absorb burst traffic	Queue depth, p99 latency at 3x normal
OOM threshold	KV cache exhaustion causes cascading failures	Max concurrent users before 503s

A.1 Locust Installation and Basic File Structure

```
pip install locust

# Run the web UI (navigate to http://localhost:8089)
locust -f locustfile.py --host http://localhost:8000

# Headless mode (for CI / automated runs)
locust -f locustfile.py --host http://localhost:8000 \
```

```
--users 100 \
--spawn-rate 5 \
--run-time 5m \
--headless \
--csv results/load_test
```

A.2 Production LLM Locust File

```
# locustfile.py -- comprehensive LLM load test
import json, time, random
from locust import HttpUser, task, between, events
from locust.runners import MasterRunner

# Realistic prompt distribution sampled from production traffic analysis
SHORT_PROMPTS = [
    "What is KV caching?",
    "Define TTFT.",
    "What is vLLM?",
    "Explain quantization in one sentence.",
]
MEDIUM_PROMPTS = [
    "Explain the difference between tensor parallelism and pipeline parallelism "
    "in the context of serving a 70B LLM.",
    "What are the three components of VRAM usage in a vLLM deployment and "
    "how would you reduce each one?",
    "Describe the PagedAttention algorithm and explain why it reduces "
    "memory fragmentation compared to naive KV cache allocation.",
]
LONG_PROMPTS = [
    ("You are an AI infrastructure expert. A user reports that their vLLM "
    "deployment is showing increasing p99 latency over time without an increase "
    "in request rate. The GPU utilization is around 85% and the KV cache usage "
    "is at 78%. Walk through your systematic debugging process step by step. ") * 2,
]

# Per-task token counters (accessed by custom metrics hook)
_task_stats = {"tokens_total": 0, "ttft_samples": [], "e2e_samples": []}

class LLMUser(HttpUser):
    wait_time = between(0.5, 2.0) # think time between requests

    def on_start(self):
        self.client.headers.update({
            "Authorization": "Bearer sk-prod-abc123",
            "Content-Type": "application/json",
        })

    @task(5) # weight 5: most common -- short Q&A
    def short_qa(self):
        self._chat(random.choice(SHORT_PROMPTS), max_tokens=80, label="short_qa")

    @task(3) # weight 3: medium complexity
    def medium_qa(self):
        self._chat(random.choice(MEDIUM_PROMPTS), max_tokens=150, label="medium_qa")

    @task(1) # weight 1: long reasoning (rare but expensive)
```

```

def long_reasoning(self):
    self._chat(random.choice(LONG_PROMPTS), max_tokens=300, label="long_reasoning")

@task(2) # weight 2: streaming requests
def streaming_qa(self):
    self._stream_chat(random.choice(MEDIUM_PROMPTS), label="streaming")

def _chat(self, prompt, max_tokens=150, label="chat"):
    payload = {
        "model": "meta-llama/Meta-Llama-3-8B-Instruct",
        "messages": [{"role": "user", "content": prompt}],
        "max_tokens": max_tokens,
        "temperature": 0.7,
    }
    t0 = time.perf_counter()
    with self.client.post(
        "/v1/chat/completions",
        json=payload,
        name=f"/v1/chat/completions [{label}]",
        catch_response=True,
    ) as resp:
        e2e = time.perf_counter() - t0
        if resp.status_code == 200:
            data = resp.json()
            tokens = data.get("usage", {}).get("total_tokens", 0)
            _task_stats["tokens_total"] += tokens
            _task_stats["e2e_samples"].append(e2e)
            resp.success()
        elif resp.status_code == 429:
            resp.failure(f"Rate limited: {resp.text[:80]}")
        elif resp.status_code == 503:
            resp.failure("Backend unavailable (OOM or overloaded)")
        else:
            resp.failure(f"HTTP {resp.status_code}: {resp.text[:80]}")

def _stream_chat(self, prompt, label="streaming"):
    payload = {
        "model": "meta-llama/Meta-Llama-3-8B-Instruct",
        "messages": [{"role": "user", "content": prompt}],
        "max_tokens": 150,
        "stream": True,
    }
    t0 = time.perf_counter()
    ttft = None
    with self.client.post(
        "/v1/chat/completions",
        json=payload,
        name=f"/v1/chat/completions [{label}]",
        stream=True,
        catch_response=True,
    ) as resp:
        if resp.status_code != 200:
            resp.failure(f"HTTP {resp.status_code}")
            return
        for line in resp.iter_lines():
            if line and line != b"data: [DONE]":

```

```

        raw = line.decode().rstrip("data: ")
        try:
            chunk = json.loads(raw)
            delta = chunk["choices"][0]["delta"].get("content", "")
            if delta and ttft is None:
                ttft = time.perf_counter() - t0
                _task_stats["ttft_samples"].append(ttft)
        except Exception:
            pass
    resp.success()

# Print custom stats at end of test
@events.quitting.add_listener
def on_quit(environment, **kwargs):
    samples = _task_stats["e2e_samples"]
    ttft = _task_stats["ttft_samples"]
    if samples:
        samples_sorted = sorted(samples)
        n = len(samples_sorted)
        print(f"
--- Custom LLM Metrics ---")
        print(f"Total requests      : {n}")
        print(f"Total tokens      : {_task_stats['tokens_total'],}")
        print(f"E2E p50          : {samples_sorted[n//2]:.2f}s")
        print(f"E2E p95          : {samples_sorted[int(n*0.95)]:.2f}s")
        print(f"E2E p99          : {samples_sorted[int(n*0.99)]:.2f}s")
    if ttft:
        ttft_sorted = sorted(ttft)
        n = len(ttft_sorted)
        print(f"TTFT p50          : {ttft_sorted[n//2]:.3f}s")
        print(f"TTFT p95          : {ttft_sorted[int(n*0.95)]:.3f}s")

```

A.3 Finding the Throughput Ceiling and OOM Threshold

```

# throughput_curve.py -- ramp concurrency to find system limits
# Run this BEFORE production traffic, not after

```

```

import subprocess, time, json, requests
import matplotlib.pyplot as plt # pip install matplotlib

```

```

BASE_URL = "http://localhost:8000"
CONCURRENCY_STEPS = [5, 10, 20, 30, 50, 75, 100, 150, 200]
DURATION_SECS = 60 # each step
RESULTS = []

```

```

for n_users in CONCURRENCY_STEPS:
    print(f"
--- Testing concurrency={n_users} ---")

```

```

    # Run Locust headless for DURATION_SECS
    result = subprocess.run([
        "locust", "-f", "locustfile.py",
        "--host", BASE_URL,
        "--users", str(n_users),
        "--spawn-rate", str(n_users),
        "--run-time", f"{DURATION_SECS}s",

```

```

        "--headless",
        "--only-summary",
        "--json",
    ], capture_output=True, text=True)

# Parse Locust JSON output
try:
    stats = json.loads(result.stdout)
    agg = next(s for s in stats if s["name"] == "Aggregated")
    RESULTS.append({
        "users" : n_users,
        "rps" : agg["current_rps"],
        "fail_rate" : agg["fail_ratio"],
        "p50_ms" : agg["response_time_percentile_0_5"],
        "p95_ms" : agg["response_time_percentile_0_95"],
        "p99_ms" : agg["response_time_percentile_0_99"],
    })
    print(f" RPS={agg['current_rps']:.1f} "
          f"p50={agg['response_time_percentile_0_5']}ms "
          f"fail={agg['fail_ratio']:.1%}")
except Exception as e:
    print(f" Parse error: {e}")

# Stop if error rate spikes (OOM / overload threshold detected)
if RESULTS and RESULTS[-1]["fail_rate"] > 0.05:
    print(f"OOM/overload threshold detected at {n_users} concurrent users")
    break

# Print summary table
print(f"
{'Users':>8} {'RPS':>8} {'p50 ms':>8} {'p95 ms':>8} {'p99 ms':>8} {'Fail%':>8}")
print("-" * 52)
for r in RESULTS:
    print(f"{r['users']:>8} {r['rps']:>8.1f} {r['p50_ms']:>8} "
          f"{r['p95_ms']:>8} {r['p99_ms']:>8} {r['fail_rate']:>8.1%}")

```

A.4 vLLM benchmark_serving.py -- Rigorous Throughput Measurement

```

# vLLM's built-in benchmark tool -- the standard for throughput reporting
# Uses the ShareGPT dataset for realistic prompt/response length distribution

# Download ShareGPT dataset
wget https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_split.json

# Benchmark at different request rates
for rate in 1 2 5 10 20 inf; do
    echo "=== Request rate: $rate req/s ==="
    python benchmark_serving.py \
        --backend vllm \
        --host localhost --port 8000 \
        --model meta-llama/Meta-Llama-3-8B-Instruct \
        --dataset-name sharegpt \
        --dataset-path ShareGPT_V3_unfiltered_cleaned_split.json \
        --num-prompts 200 \
        --request-rate $rate \
        --percentile-metrics ttft,tpot,itl,e2el \

```

```

        --save-result \
        --result-dir ./benchmark_results
done

# Plot tokens/sec vs concurrency curve from results
python -c "
import json, glob, matplotlib.pyplot as plt

results = []
for f in sorted(glob.glob('./benchmark_results/*.json')):
    with open(f) as fh:
        d = json.load(fh)
        results.append({
            'rate'      : d.get('request_rate', 0),
            'tps'       : d.get('output_throughput', 0),
            'ttft_p50'  : d.get('mean_ttft_ms', 0),
            'ttft_p99'  : d.get('p99_ttft_ms', 0),
        })

results.sort(key=lambda x: x['rate'])
rates      = [r['rate'] for r in results]
tps        = [r['tps']  for r in results]
ttft_p99   = [r['ttft_p99'] for r in results]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.plot(rates, tps, 'b-o'); ax1.set_xlabel('Request rate (req/s)')
ax1.set_ylabel('Output throughput (tok/s)'); ax1.set_title('Throughput curve')
ax2.plot(rates, ttft_p99, 'r-o'); ax2.set_xlabel('Request rate (req/s)')
ax2.set_ylabel('TTFT p99 (ms)'); ax2.set_title('Latency curve')
plt.tight_layout(); plt.savefig('benchmark_curves.png', dpi=150)
print('Saved benchmark_curves.png')
"

```

[illegible]


```

environment:
  - COLLECTOR_OTLP_ENABLED=true
ports:
  - "16686:16686"    # Jaeger UI
  - "4317:4317"     # OTLP gRPC receiver
  - "4318:4318"     # OTLP HTTP receiver
volumes:
  - jaeger_data:/badger

volumes:
  jaeger_data:

# Start: docker compose -f docker-compose.observability.yml up -d
# UI:    http://localhost:16686
# Search for service "llm-gateway" and filter by operation "vllm_inference"

# Grafana Tempo (production alternative -- integrates with Grafana dashboards)
tempo:
  image: grafana/tempo:latest
  command: ["-config.file=/etc/tempo.yaml"]
  volumes:
    - ./tempo.yaml:/etc/tempo.yaml
  ports:
    - "3200:3200"    # Tempo query frontend
    - "4317:4317"    # OTLP gRPC

```

B.3 vLLM Native OpenTelemetry Support

```

# vLLM 0.4+ ships with built-in OpenTelemetry instrumentation
# Enable by setting environment variables before starting the server

OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317 \
OTEL_SERVICE_NAME=vllm-inference \
OTEL_EXPORTER_OTLP_INSECURE=true \
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
  --dtype float16 \
  --otlp-traces-endpoint http://jaeger:4317 \
  --collect-detailed-traces all # "all" | "generate" | "engine"

# vLLM traces export these spans automatically:
# - llm_request           : full request lifecycle
# - vllm.scheduler        : time waiting in queue
# - vllm.executor         : time in GPU forward passes
#   - vllm.prefill        : prompt processing phase
#   - vllm.decode         : token generation phase
# - vllm.postprocess      : response serialisation

# Attributes on vllm.prefill span:
#   vllm.num_prompt_tokens
#   vllm.num_cached_tokens    <- from prefix cache
#   vllm.time_in_queue_ms

# Attributes on vllm.decode span:
#   vllm.num_generated_tokens
#   vllm.time_per_output_token_ms

```

PART C — Loki Log Aggregation

The Loki Stack: Promtail + Loki + Grafana

Loki is Grafana's log aggregation system. Unlike Elasticsearch, Loki does not index log content -- it only indexes labels (key-value metadata). This makes it dramatically cheaper to operate at high log volume. Logs are queried using LogQL, which is intentionally similar to PromQL. The three-component stack: Promtail (log shipper, runs as a DaemonSet) collects container logs, ships them to Loki (storage + query engine), which is queried from Grafana dashboards alongside Prometheus metrics.

C.1 Deploy the Loki Stack with Docker Compose

```
# docker-compose.loki.yml
version: "3.8"
services:

  loki:
    image: grafana/loki:2.9.0
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml
    volumes:
      - loki_data:/loki

  promtail:
    image: grafana/promtail:2.9.0
    volumes:
      - /var/log:/var/log:ro
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
      - ./promtail-config.yaml:/etc/promtail/config.yaml
    command: -config.file=/etc/promtail/config.yaml
    depends_on: [loki]

  grafana:
    image: grafana/grafana:10.2.0
    ports:
      - "3000:3000"
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana-provisioning:/etc/grafana/provisioning
    depends_on: [loki, prometheus]

volumes:
  loki_data:
  grafana_data:
```

C.2 Promtail Config for LLM Gateway Logs

```
# promtail-config.yaml
server:
  http_listen_port: 9080
```

```

    grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  # Scrape vLLM container logs
  - job_name: vllm
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
        filters:
          - name: label
            values: ["com.docker.compose.service=vllm-primary"]
    relabel_configs:
      - source_labels: [__meta_docker_container_name]
        target_label: container
      - source_labels: [__meta_docker_compose_service]
        target_label: service
    pipeline_stages:
      # Parse JSON structured logs from vLLM
      - json:
          expressions:
            level: level
            message: msg
            model: model
            tokens: total_tokens
            latency: elapsed_s
      - labels:
          level:
          service:
      - metrics:
          tokens_per_request:
            type: Histogram
            description: "Token count per request"
            source: tokens
            config:
              buckets: [50, 100, 200, 500, 1000, 2000]

  # Scrape gateway structured JSON logs
  - job_name: llm-gateway
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        filters:
          - name: label
            values: ["com.docker.compose.service=llm-gateway"]
    pipeline_stages:
      - json:
          expressions:
            api_key: api_key
            backend: backend
            tokens: tokens
            latency_ms: latency_ms

```

```
- labels:  
  backend:
```

C.3 LogQL Queries for LLM Debugging

```
# LogQL reference -- run these in Grafana Explore or Loki API  
  
# 1. All vLLM errors in last 1 hour  
{service="vllm-primary"} |= "ERROR" | json | line_format "{{.message}}"  
  
# 2. Requests with token count > 1000 (expensive requests)  
{service="llm-gateway"  
  | json  
  | tokens > 1000  
  | line_format "api={{.api_key}} tokens={{.tokens}} lat={{.latency_ms}}ms"  
  
# 3. Error rate over time (rate of 5xx responses)  
rate({service="llm-gateway"} |= "status=5" [5m])  
  
# 4. p95 latency from logs (using quantile_over_time)  
quantile_over_time(0.95,  
  {service="llm-gateway"  
    | json  
    | unwrap latency_ms [5m]  
  )  
  
# 5. Top API keys by token consumption (last 24h)  
topk(10,  
  sum by (api_key) (  
    increase(  
      {service="llm-gateway"} | json | unwrap tokens [24h]  
    )  
  )  
)  
  
# 6. Cache hit rate from gateway logs  
sum(rate({service="llm-gateway"} | json | cache_hit="true" [5m]))  
/  
sum(rate({service="llm-gateway"} | json [5m]))  
  
# 7. OOM events (vLLM CUDA out of memory)  
{service="vllm-primary"} |~ "CUDA out of memory|OutOfMemoryError"  
| line_format "OOM at {{.__timestamp__}}: {{.message}}"
```

PART D — Token-Level Cost Tracking

D.1 Cost Tracking Middleware

```
# cost_middleware.py -- FastAPI middleware for per-request cost attribution
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
from prometheus_client import Counter, Histogram, Gauge
import json, time

# Prometheus metrics
PROMPT_TOKENS = Counter("llm_prompt_tokens_total",
                        "Total prompt tokens", ["model", "api_key_prefix", "route"])
COMPLETION_TOKENS = Counter("llm_completion_tokens_total",
                            "Total completion tokens", ["model", "api_key_prefix", "route"])
COST_DOLLARS = Counter("llm_cost_dollars_total",
                      "Total inferred cost in USD", ["model", "api_key_prefix"])
REQUEST_LATENCY = Histogram("llm_request_latency_seconds",
                             "Request latency", ["model", "status"],
                             buckets=[0.1, 0.5, 1, 2, 5, 10, 30, 60])

# Cost per 1K tokens by model (self-hosted GPU cost estimate)
MODEL_COST_PER_1K = {
    "meta-llama/Meta-Llama-3-8B-Instruct" : 0.0004,    # ~$0.40/M tokens (A10G)
    "meta-llama/Meta-Llama-3-70B-Instruct": 0.002,    # ~$2.00/M tokens (A100x4)
    "microsoft/phi-2"                    : 0.0001,    # very cheap (small model)
}

class CostTrackingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
        t0 = time.perf_counter()
        response = await call_next(request)
        elapsed = time.perf_counter() - t0

        # Only track /v1/chat/completions and /v1/completions
        if not request.url.path.startswith("/v1/"):
            return response

        api_key = (request.headers.get("Authorization", "")
                  .removeprefix("Bearer ")[:8])
        route = request.url.path

        # Read response body to extract token usage
        body = b""
        async for chunk in response.body_iterator:
            body += chunk

        try:
            data = json.loads(body)
            usage = data.get("usage", {})
            model = data.get("model", "unknown")
            p_toks = usage.get("prompt_tokens", 0)
            c_toks = usage.get("completion_tokens", 0)
            status = str(response.status_code)

            cost_per_1k = MODEL_COST_PER_1K.get(model, 0.001)
```

```

        cost            = (p_toks + c_toks) / 1000 * cost_per_1k

        PROMPT_TOKENS.labels(model, api_key, route).inc(p_toks)
        COMPLETION_TOKENS.labels(model, api_key, route).inc(c_toks)
        COST_DOLLARS.labels(model, api_key).inc(cost)
        REQUEST_LATENCY.labels(model, status).observe(elapsed)

    except Exception:
        pass    # don't break response if parsing fails

    from fastapi.responses import Response as FastResponse
    return FastResponse(
        content=body,
        status_code=response.status_code,
        headers=dict(response.headers),
        media_type=response.media_type,
    )

# Register middleware in FastAPI app
# app.add_middleware(CostTrackingMiddleware)

```

D.2 Grafana Cost Dashboard Panel Queries (PromQL)

```

# Paste these into Grafana Explore or a dashboard panel (data source: Prometheus)

# 1. Hourly cost per model (USD)
increase(llm_cost_dollars_total[1h])

# 2. Cost per API key (top 10, last 24h)
topk(10, increase(llm_cost_dollars_total[24h]))

# 3. Tokens per second (prompt + completion, rolling 1m)
sum(rate(llm_prompt_tokens_total[1m])) +
sum(rate(llm_completion_tokens_total[1m]))

# 4. Prompt-to-completion token ratio (efficiency indicator)
# High ratio = users sending large prompts, low generation -- possible RAG issue
sum(rate(llm_prompt_tokens_total[5m]))
/
sum(rate(llm_completion_tokens_total[5m]))

# 5. Cost rate: current spend in $/hour
sum(rate(llm_cost_dollars_total[5m])) * 3600

# 6. Request latency p95 by model
histogram_quantile(0.95,
    sum by (le, model) (rate(llm_request_latency_seconds_bucket[5m]))
)

# 7. Daily projected cost (extrapolate from current rate)
sum(rate(llm_cost_dollars_total[1h])) * 24

```

PART E — A/B Testing LLM Responses

Why A/B Test LLMs?

Upgrading a model (e.g. 8B -> 70B, or base -> fine-tuned) without a controlled traffic split is a binary risk: you either deploy to 100% of users or to nobody. A/B testing routes a small percentage of traffic (e.g. 10%) to the candidate model while the rest continues on the control model. Quality metrics (LLM-as-judge scores, user ratings, task success rates) are compared between variants before full rollout.

E.1 Traffic Splitting in the Gateway

```
# ab_router.py -- route X% of traffic to model B, rest to model A
import hashlib, time
from openai import AsyncOpenAI
from fastapi import Request
import json

MODEL_A = "meta-llama/Meta-Llama-3-8B-Instruct" # control
MODEL_B = "meta-llama/Meta-Llama-3-70B-Instruct" # candidate

TRAFFIC_B_PCT = 0.10 # 10% to model B

client_a = AsyncOpenAI(base_url="http://vllm-8b:8000/v1", api_key="vllm")
client_b = AsyncOpenAI(base_url="http://vllm-70b:8000/v1", api_key="vllm")

def assign_variant(api_key: str, experiment: str = "model_ab") -> str:
    # Deterministic assignment: same user always gets same variant
    # Uses hash(api_key + experiment) to ensure sticky assignment
    seed = hashlib.md5(f"{api_key}:{experiment}".encode()).hexdigest()
    bucket = int(seed[:8], 16) / 0xFFFFFFFF # 0.0 - 1.0
    return "B" if bucket < TRAFFIC_B_PCT else "A"

async def ab_completion(messages, api_key, max_tokens=200):
    variant = assign_variant(api_key)
    client = client_b if variant == "B" else client_a
    model = MODEL_B if variant == "B" else MODEL_A

    t0 = time.perf_counter()
    resp = await client.chat.completions.create(
        model=model, messages=messages,
        max_tokens=max_tokens, temperature=0.7,
    )
    elapsed = time.perf_counter() - t0

    # Log variant assignment for analysis
    print(json.dumps({
        "experiment": "model_ab",
        "variant" : variant,
        "model" : model,
        "api_key" : api_key[:8],
        "tokens" : resp.usage.total_tokens,
        "latency_ms": round(elapsed * 1000),
    }))

    return resp, variant
```

E.2 LLM-as-Judge Quality Scoring

```
# Score responses from variant A and B using a strong judge model
# The judge evaluates: accuracy, completeness, conciseness (1-5 scale each)

from openai import OpenAI
import json, statistics

judge_client = OpenAI(base_url="http://judge-model:8000/v1", api_key="vllm")

JUDGE_SYSTEM = (
    "You are an expert evaluator. Score the assistant response on three dimensions "
    "from 1 (very poor) to 5 (excellent). Reply ONLY with valid JSON in this format: "
    '{"accuracy": N, "completeness": N, "conciseness": N, "reasoning": "..."}'
)

def judge_response(question: str, response: str) -> dict:
    user_msg = f"Question: {question}"

    Response to evaluate:
    {response}"
    resp = judge_client.chat.completions.create(
        model="meta-llama/Meta-Llama-3-70B-Instruct",
        messages=[
            {"role": "system", "content": JUDGE_SYSTEM},
            {"role": "user", "content": user_msg},
        ],
        max_tokens=200, temperature=0.0,
    )
    try:
        return json.loads(resp.choices[0].message.content)
    except Exception:
        return {"accuracy": 0, "completeness": 0, "conciseness": 0, "reasoning": "parse error"}

# Collect scores from both variants
def compare_variants(test_questions: list, n_per_question=5) -> dict:
    scores = {"A": [], "B": []}

    for q in test_questions:
        for variant, client, model in [
            ("A", client_a, MODEL_A),
            ("B", client_b, MODEL_B),
        ]:
            resp = client.chat.completions.create(
                model=model,
                messages=[{"role": "user", "content": q}],
                max_tokens=200, temperature=0.7,
            )
            text = resp.choices[0].message.content
            score = judge_response(q, text)
            total = (score["accuracy"] + score["completeness"] + score["conciseness"]) / 3
            scores[variant].append(total)
            print(f" [{variant}] Q: {q[:40]}... Score: {total:.2f}")

    return {
        "A_mean": statistics.mean(scores["A"]),
    }
```

```

        "B_mean": statistics.mean(scores["B"]),
        "A_stdev": statistics.stdev(scores["A"]) if len(scores["A"]) > 1 else 0,
        "B_stdev": statistics.stdev(scores["B"]) if len(scores["B"]) > 1 else 0,
        "delta": statistics.mean(scores["B"]) - statistics.mean(scores["A"]),
    }

TEST_QS = [
    "Explain PagedAttention in detail.",
    "What are the trade-offs between tensor and pipeline parallelism?",
    "How would you debug high p99 latency in a vLLM deployment?",
    "Describe a production-ready LLM serving architecture.",
]

results = compare_variants(TEST_QS)
print(f"
--- A/B Quality Comparison ---")
print(f"Model A (control): {results['A_mean']:.3f} +/- {results['A_stdev']:.3f}")
print(f"Model B (candidate): {results['B_mean']:.3f} +/- {results['B_stdev']:.3f}")
print(f"Delta: {results['delta']:+.3f}")
rollout = "RECOMMEND ROLLOUT" if results["delta"] > 0.2 else "HOLD -- delta too small"
print(f"Decision: {rollout}")

```

Testing & Metrics Reference

Full Observability Stack -- Component Summary

Signal	Tool	What it covers	Key query/command
Metrics	Prometheus	Throughput, latency histograms, GPU util, cost	cost_histogram_quantile(0.95, ...)
Logs	Loki+Promtail	Structured JSON per request, errors, OOM events	avg_rate(({service='vllm'}) [5m])
Traces	Jaeger/Tempo	Per-request span breakdown: cache/prefix/detoken by service + operation + duration	End-to-end
Load test	Locust	Throughput ceiling, OOM threshold, latency under load	locust --spawn-rate 5
Benchmark	vLLM bench	Rigorous tok/s vs request-rate curve	benchmark_serving.py --request-rate inf
A/B	Gateway split	Quality delta between model variants	assign_variant() + LLM-as-judge
Cost	Prometheus	USD/hr per model per API key	rate(llm_cost_dollars_total[1h])*3600

Production Observability Checklist

```

# Before going to production, verify each signal:

# 1. Metrics: hit /metrics and verify core vLLM metrics present
curl http://localhost:8000/metrics | grep -c "vllm:"
# Should be > 20

# 2. Logs: confirm structured JSON logging from gateway
curl -s http://localhost:80/v1/chat/completions \
  -H "Authorization: Bearer sk-prod-abc123" \
  -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"hi"}], "max_tokens":5}'

```

```

docker logs llm-gateway 2>&1 | tail -3
# Should show: {"ts":..., "api_key":..., "backend":..., "tokens":...}

# 3. Traces: verify spans appear in Jaeger
curl http://localhost:16686/api/services | python -m json.tool
# Should include: "llm-gateway", "vllm-inference"

# 4. Cost tracking: fire a request and check Prometheus counter
curl -s http://localhost:9090/api/v1/query \
  --data-urlencode 'query=llm_cost_dollars_total' | python -m json.tool

# 5. A/B routing: fire 20 requests, check ~10% get routed to variant B
for i in $(seq 20); do
  curl -s http://localhost:80/v1/chat/completions \
    -H "Authorization: Bearer sk-prod-user-$RANDOM" \
    -H "Content-Type: application/json" \
    -d '{"messages":[{"role":"user","content":"hi"}], "max_tokens":5}' \
    -o /dev/null
done
docker logs llm-gateway 2>&1 | tail -20 | grep variant | sort | uniq -c
# Expected: ~18x "A", ~2x "B"

```

Key Takeaways

- **Load test before production.** Run the throughput ceiling script to find the exact concurrency at which error rate spikes -- this is your KV cache OOM threshold. Set HPA max replicas at 70% of that number.
- **Locust models realistic traffic distributions:** weight tasks by expected frequency (short:medium:long = 5:3:1) and include streaming requests as a separate task. p99 latency under mixed load is what matters.
- **OpenTelemetry gives you the span breakdown** that metrics cannot: which of cache lookup, queue wait, prefill, and decode is your bottleneck. Without traces, p99 spikes are nearly impossible to diagnose.
- **Loki + structured JSON logs** enable cost attribution and error analysis at high volume with low storage cost. Always log: api_key, model, tokens, latency_ms, backend, cache_hit as structured fields.
- **Token-level cost tracking via Prometheus** gives you a live \$/hr dashboard. The most valuable derived metric is cost per API key per hour -- it surfaces runaway clients and mis-routed expensive queries immediately.
- **A/B test every model change.** Deterministic hash-based variant assignment ensures sticky routing (same user always gets same model) which prevents confounds in quality scores. LLM-as-judge scoring on 3-5 dimensions is more actionable than a single rating.

Next topic: **Day 14 (Capstone) -- Orchestration & MLOps:** Airflow DAG for data ingest -> eval -> promote, KubeFlow pipeline for fine-tune -> eval -> serve, LoRA adapter deployment with vLLM hot-swap, and continuous fine-tuning pipelines with automated quality gates.